

# Introduction aux API

- Comprendre la notion de service web et son utilité
- Appréhender la documentation d'un service web
- Interagir avec une API à travers un logiciel adapté
- Être capable d'interagir avec les équipes techniques

## **I. Introduction**

1. Le problème
- 

## **II. Concepts phares**

1. Orienté services
  2. Application Programming Interface
  3. Approche microservices
- 

## **III. Interagir avec une API**

1. Le modèle réponse-requête
  2. Postman : un outil
  3. Endpoint et méthode
- 

## **IV. Documenter, concevoir, spécifier**

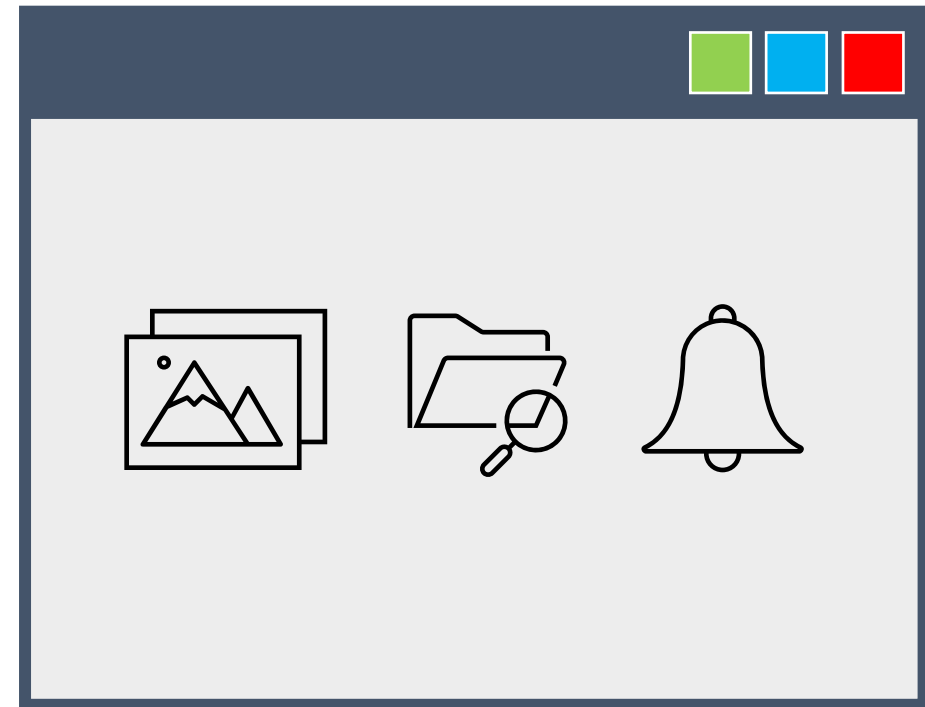
1. Lire une documentation
2. Concevoir et documenter

# Introduction

L'entreprise ACME utilise l'appli Gestion+ dans le cadre de son activité depuis 1996.

C'est une application fiable, appréciée par les collaborateurs ACME, et qui permet à l'entreprise de livrer des produits de qualité à ses clients.

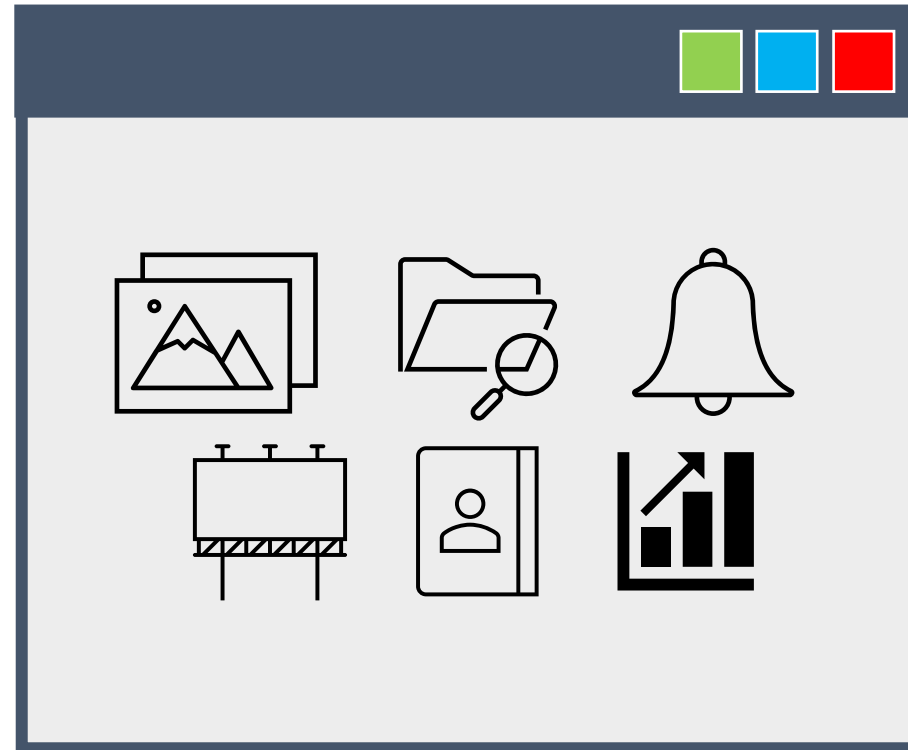
C'est une application simple, environ 22.000 lignes de code, qui utilise les derniers standards en matière de langage de programmation.



Gestion+ v1.0

Un nouveau besoin? Simple, on rajoute des lignes de code.

Suite à des modifications de l'activité de l'entreprise, il a fallu intégrer de nouvelles fonctionnalités entre 1997 et 2004...



Gestion+ v1.6

... le code de l'application devenant chaque fois plus difficile à comprendre et à gérer.

Il y a de fortes dépendances entre les fonctionnalités, et les nouveaux développements entraînent des régressions sur l'existant.

2005, c'est pour ACME l'année de tous les succès. Elle triple son nombre de clients et double son CA.

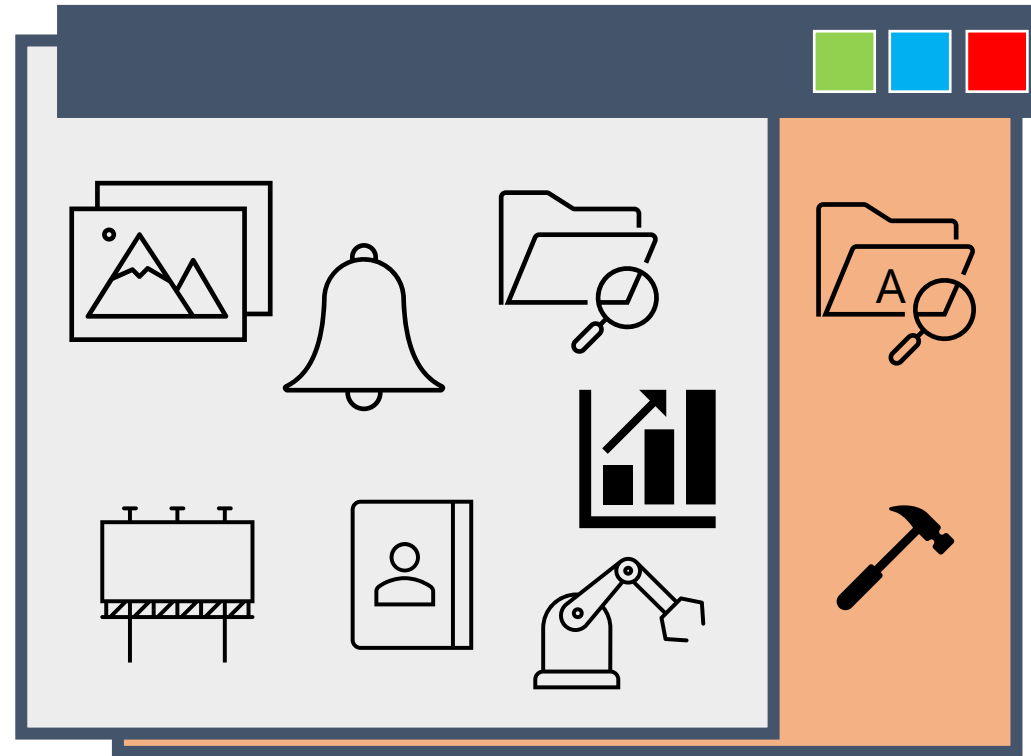
Problème, Gestion+ n'a jamais été pensée pour traiter autant de clients.

De mars à juin, impossible de créer un nouveau dossier client.

Le DSI, dans l'urgence, hésite à abandonner l'application et à utiliser une solution du marché, mais recule devant les coûts investis dans le développement de Gestion+.

La solution sera finalement résolue en appliquant des patches successifs, mais l'application conservera de cette époque des lenteurs de traitement.

Le rachat de l'entreprise AJAX en 2006 n'a pas aidé.



La MOE ne sait plus dire  
combien de lignes de  
code composent  
l'application.

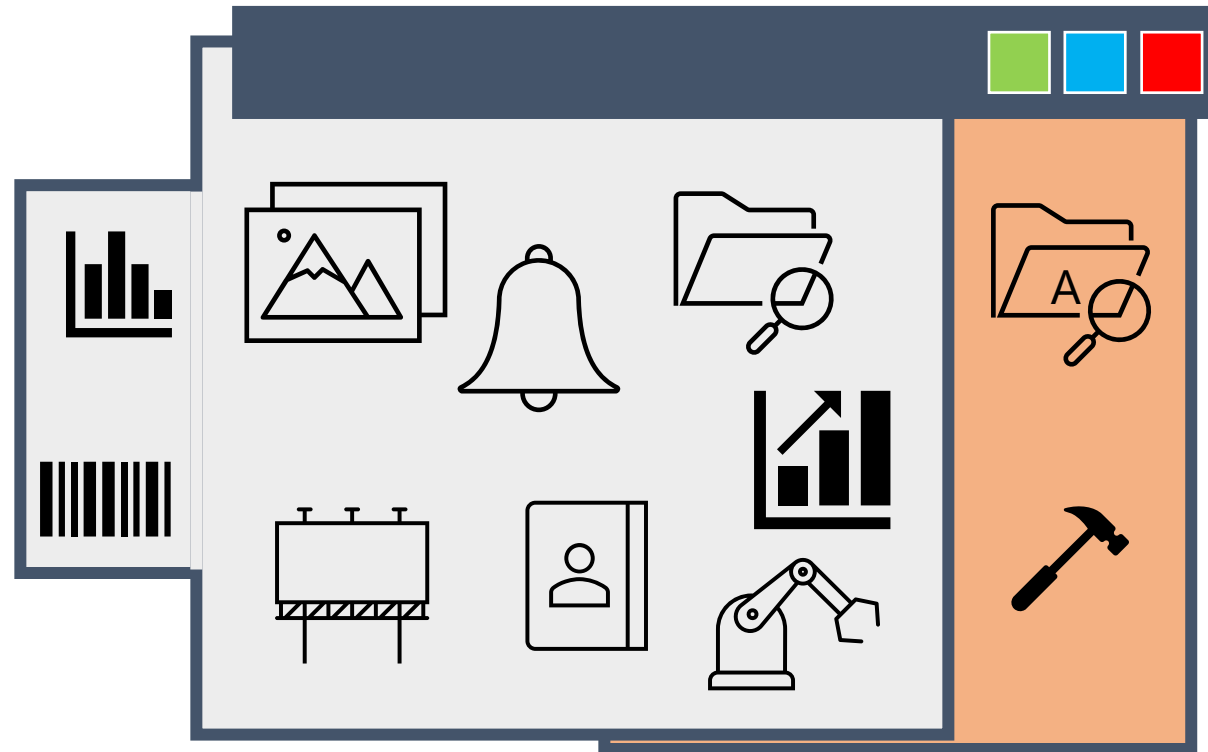


Cf. Wikipedia :  
[Syndrome du plat de spaghettis](#)

Gestion+ v2.3



Ni la récupération des activités de la filiale allemande en 2007.



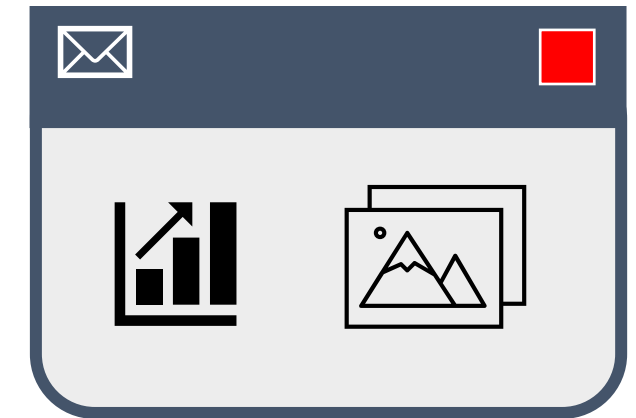
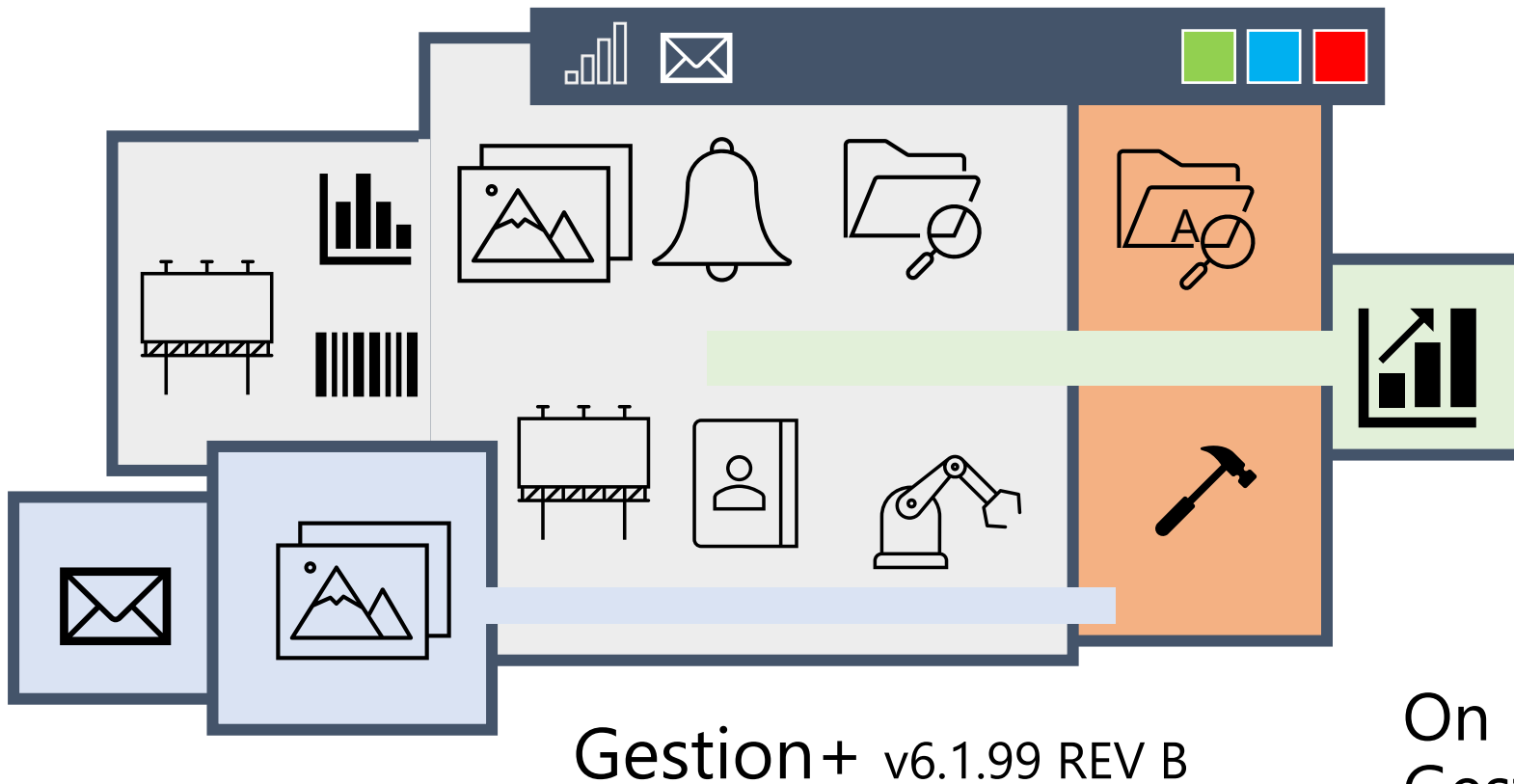
Gestion+ v2.4.45

Le jour où l'éditeur du système de gestion de bases de données sur lequel est basée Gestion+ a annoncé arrêter le support technique, le DSI ACME a ricané, est sorti de son bureau et n'est plus jamais venu travailler.

Le nouveau DSI, arrivé en 2008, veut lancer un gros projet de remise à niveau de l'application par rapport aux besoins actuels de ses utilisateurs. Problème, la technologie sur laquelle est basée l'application ne permet pas de mettre en place facilement les fonctionnalités en question.

Le projet est abandonné.

En 2008, l'entreprise décide d'étendre son SI avec la prise en compte de différents navigateurs web, puis, en 2010 aux téléphones et aux tablettes.



Mobi+ v1.2.19 REV A

On recrée des morceaux de Gestion+ dans une appli mobile.

Les utilisateurs sont de moins en moins satisfaits : manque de souplesse de l'application, traitements lourds, lenteurs, des bugs qui traînent des années sans être résolus...

Les développeurs sont frustrés, passent trop de temps à faire de la lecture de code, de l'analyse, et de la maintenance, et trop peu de temps à programmer.

En 2013, alors qu'on cherche en vain à intégrer une fonctionnalité simple depuis deux ans et demi, il est décidé de lancer une nouvelle application sans rien garder de Gestion+.



Par rapport à la situation décrite, quels problèmes identifiez-vous?

## Problèmes identifiés :



Pour aller plus loin :  
Début vidéo Randy Shoup  
<https://youtu.be/oejXFgvAwTA>

L'architecture orienté services (SOA – service-oriented architecture) :

- Des composants simples
- Indépendants les uns des autres
- Qui ont des interactions simples
- Qu'on peut **comprendre**  
**faire évoluer** facilement et indépendamment  
**tester**  
**décommissionner**

# Concepts phares

---

- I. Orienté service
- II. API : Application Programming Interface
- III. Approche microservices



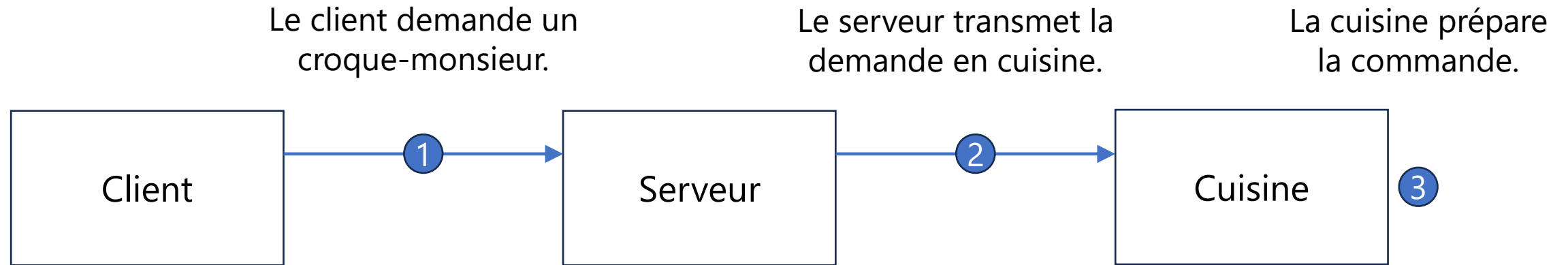
Dans un café, un client commande un croque-monsieur.

- Qui émet la demande?
- À qui le demande-t-il?
- Qui le prépare?
- Qui le lui sert?
- Qui le consomme?

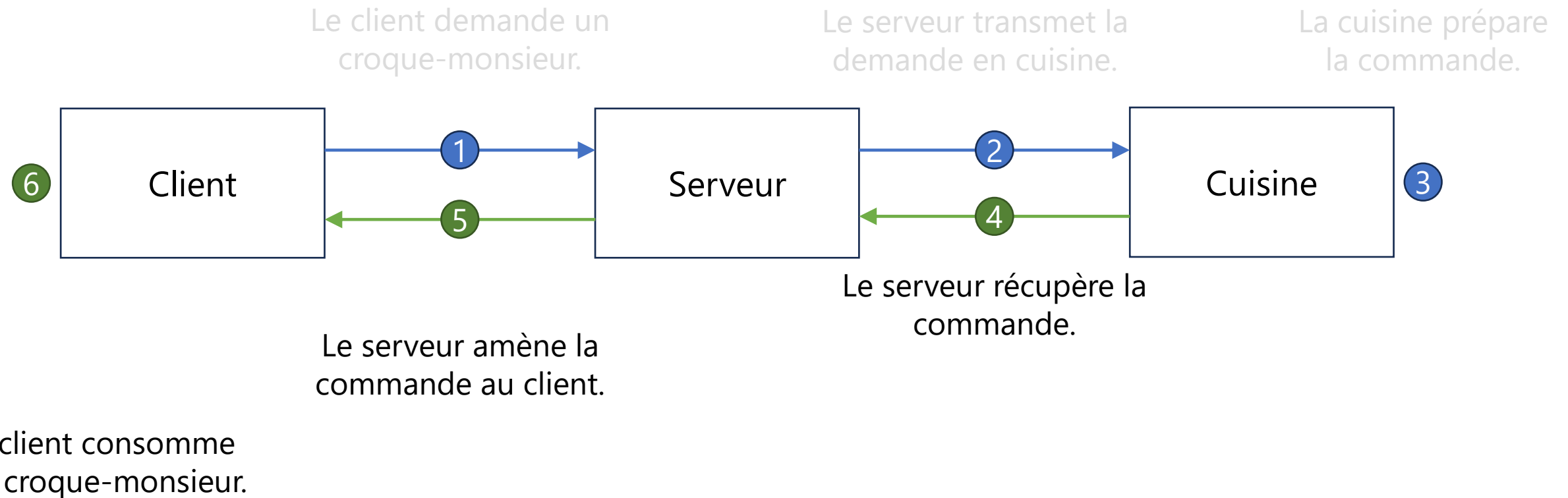


[Cette photo](#) par Auteur inconnu est soumise à la licence [CC BY-SA-NC](#)

Dans un café, un client commande un croque-monsieur.



Dans un café, un client commande un croque-monsieur.



Le **client** veut consommer une production de la **cuisine**, mais sans accéder ou parler à la cuisine, et sans toucher ni au frigo ni au four.

Il passe pour cela par une **couche de médiation**, le **service**.

C'est ce concept qui est appliqué dans les **architectures orientées services**.

- Un **service**, c'est ce qui va permettre à un consommateur de réaliser une action dans une autre application :
  - Passer une commande
  - Consulter un solde
  - Ajouter sa date de naissance sur son dossier
  - Effacer un compte client
  - ...
- Un service est en général un composant **simple**, qui fonctionne en **boîte noire** pour ses utilisateurs, et qui **cohabite** généralement avec d'autres services avec lesquels il est **faiblement couplé**.

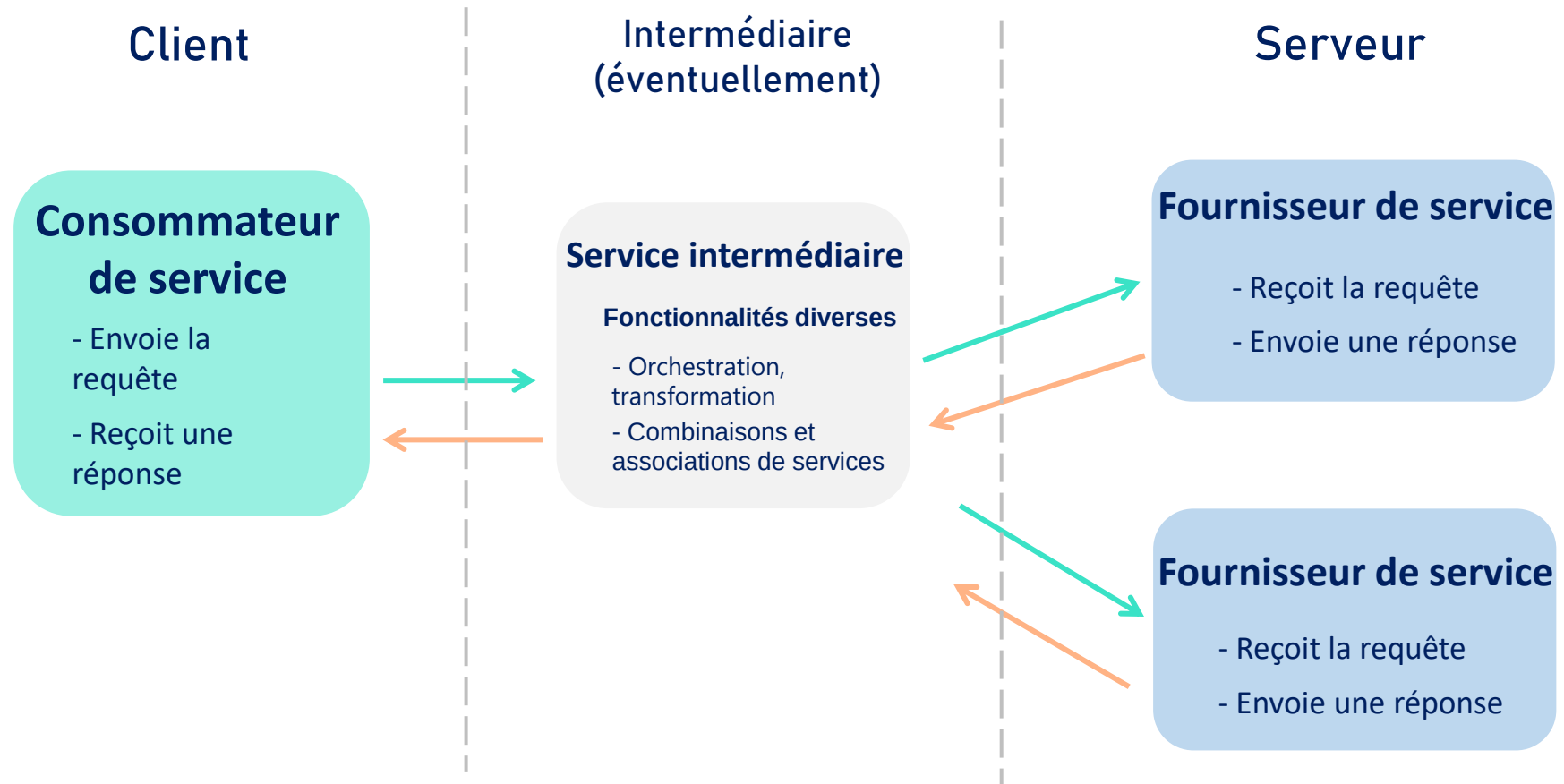
## Système monolithique

- Un système fortement couplé oblige à faire évoluer la solution tout d'un coup, en coordonnant potentiellement plusieurs équipes, et alors que la base de code est de plus en plus complexe et de moins en moins maîtrisée. L'intégration de nouvelles fonctionnalités ou de nouveaux usages est de plus en plus complexe au fur et à mesure que le monolithe grandit.

## Architecture orientée services

- Un système découpé en petits morceaux permet de rajouter ou de modifier des morceaux uns par un, tout en limitant leur complexité.
- Les équipes peuvent facilement se répartir les services sur lesquels ils travaillent.
- Cela facilite l'intégration de nouveaux consommateurs de données, qui peuvent venir d'un autre SI et utiliser un autre OS sans avoir à repenser le système.

## Consommateurs et fournisseurs de services



Ex : je veux intégrer à mon site web les taux de change en temps réel



<https://exchangeratesapi.io/>

# Application Programming Interface

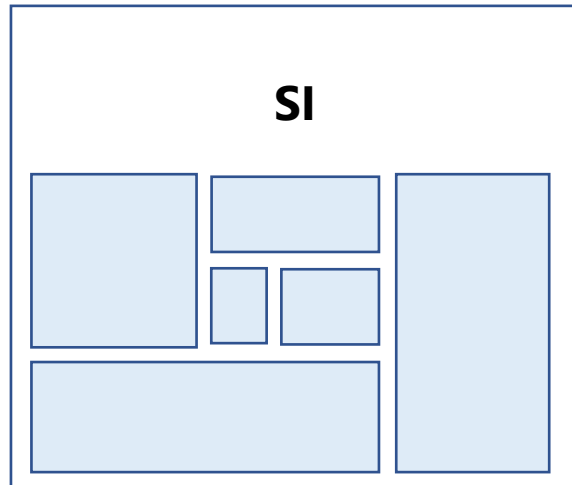
Interface de programmation applicative

Une **interface** qui va permettre à des **applications** de communiquer entre elles.

Pour aller plus loin : *"What is an API? In English, please"* par Petr Gazarov

<https://www.freecodecamp.org/news/what-is-an-api-in-english-please-b880a3214a82/>

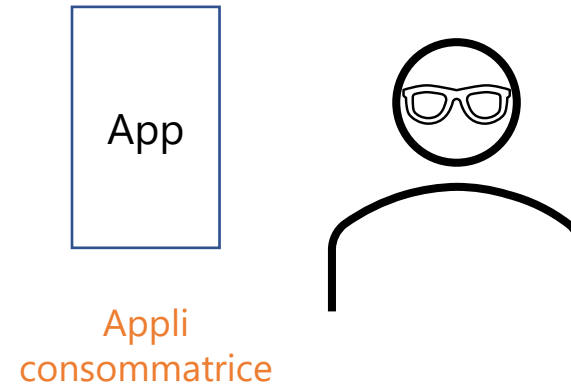


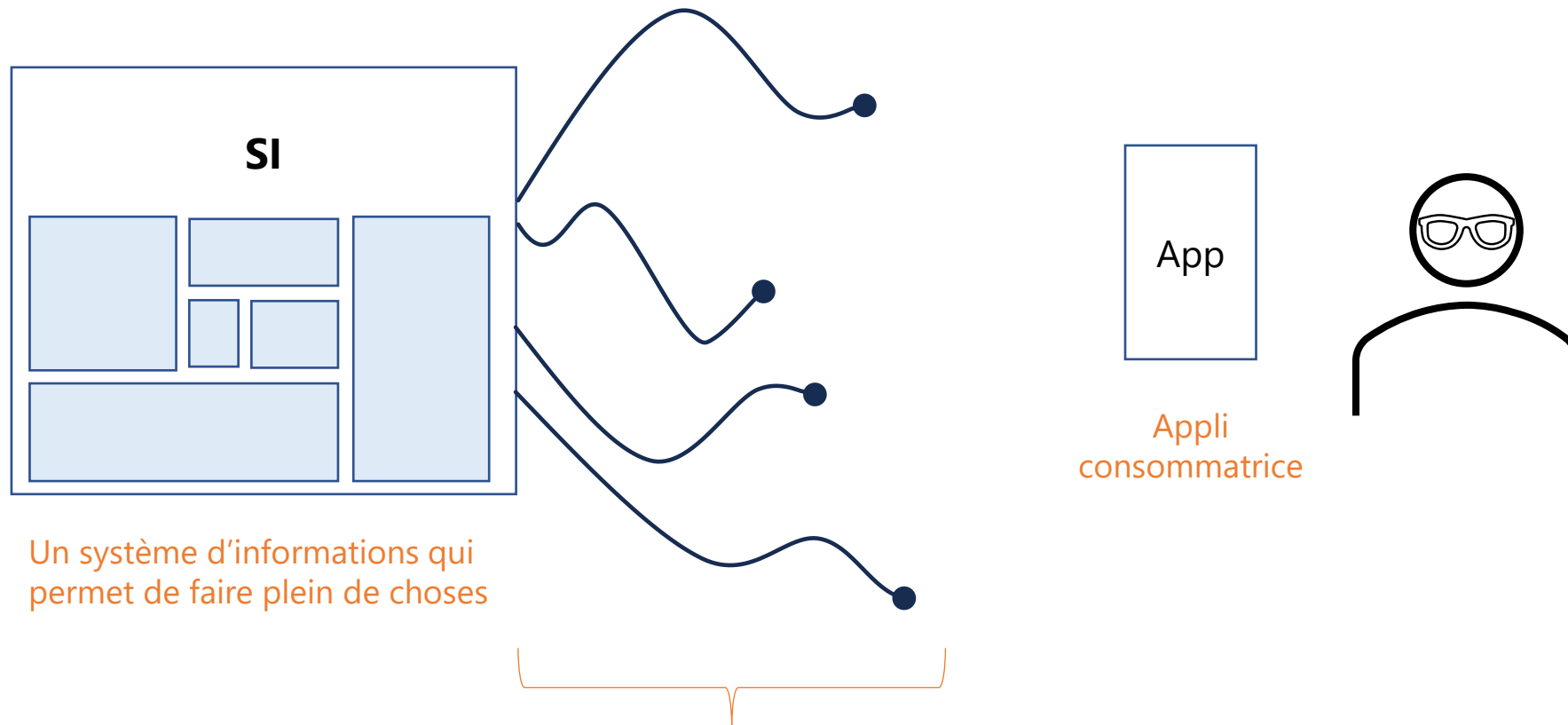


Un système d'informations qui  
permet de faire plein de choses

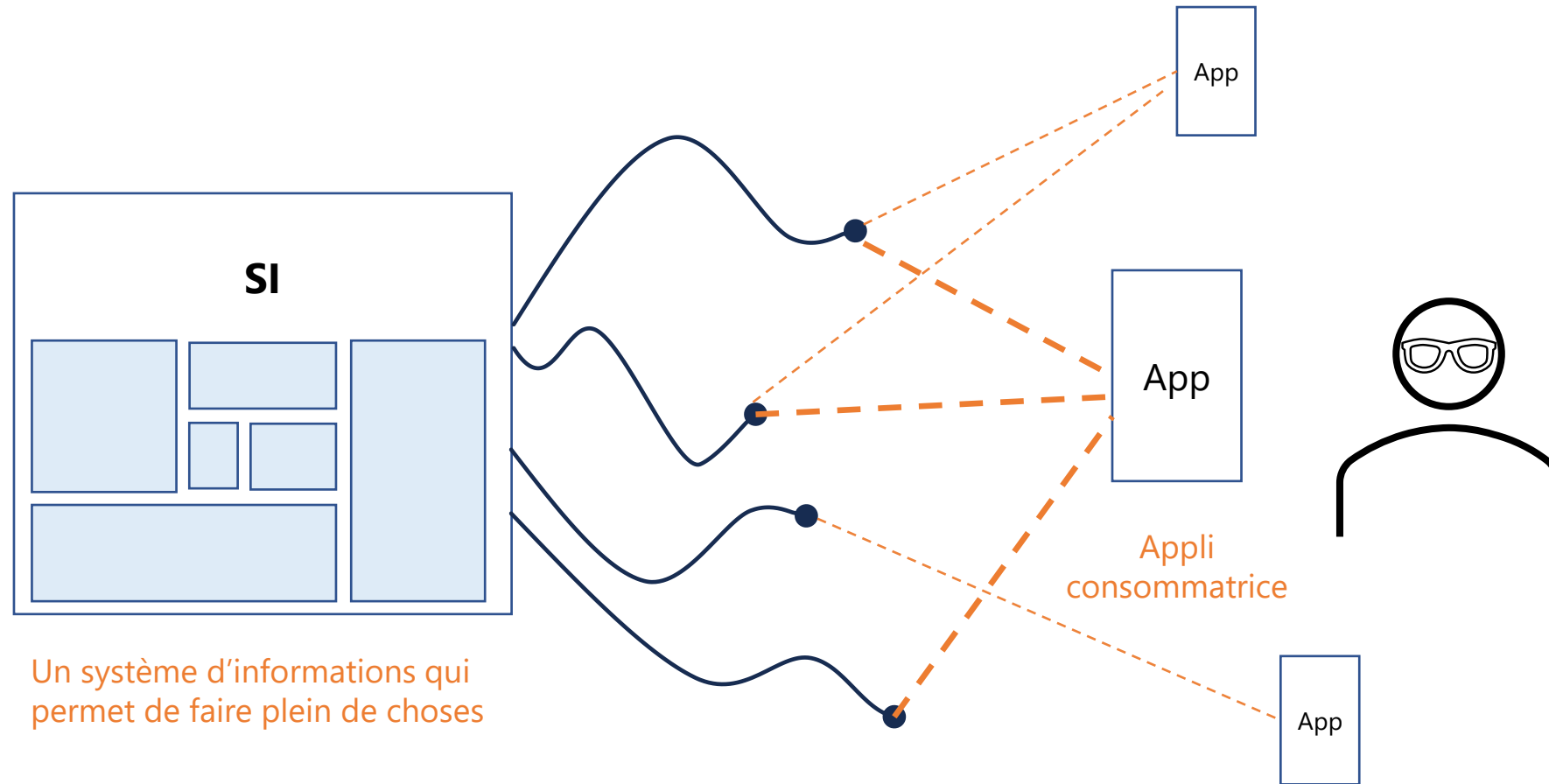
Le SI peut être complexe, être  
composé de nombreux modules,  
d'applications, de bases de  
données, faire appel à des SI  
externes, etc.

Un utilisateur a besoin d'un service rendu par le SI.





Le SI va **exposer** des services et des fonctionnalités à travers ses **API**



... qu'une ou plusieurs applications vont pouvoir **consommer**.

Connaissez-vous des API?



### API TikTok

Permet à des applications de consulter TikTok : voir les tendances, faire des recherches, lire les commentaires d'une vidéo, trouver les vidéos d'un utilisateur, identifier les publicités d'un annonceur, etc.



Twitter API



Google Search



Explore API



API Sirene



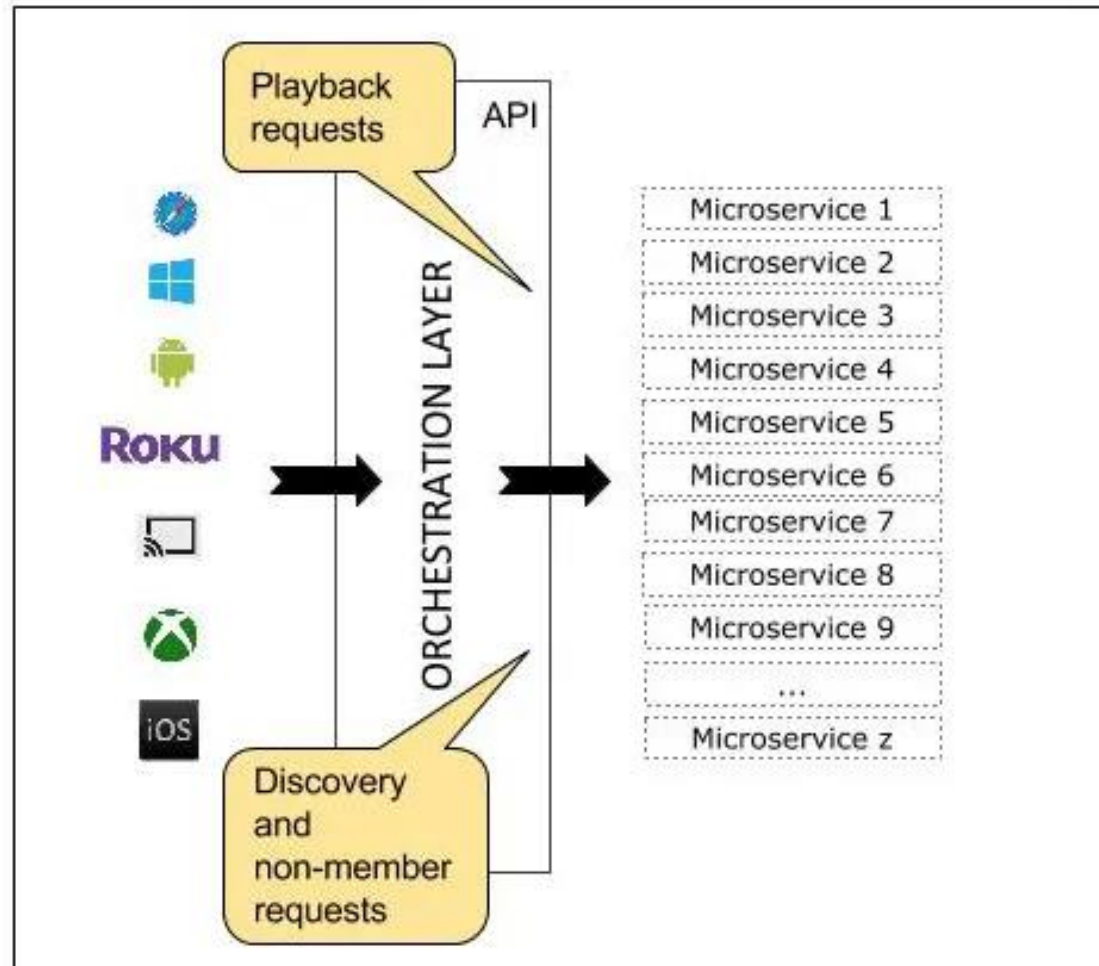
Qu'est-ce qui peut pousser une organisation à avoir des applications externes qui accèdent à son API?

- En poussant cette logique à l'extrême, on arrive à l'approche microservices : un **écosystème** de services qui font chacun **une seule** chose, sans redondances ni dépendances de code entre eux, et qui sont capables de s'appeler les uns les autres quand c'est nécessaire.
- Aujourd'hui, surtout une approche initiée par les géants de la tech (eBay, Amazon, Netflix, ...) avec l'idée de leur permettre de faire des mises en production quotidiennes.

Pour aller plus loin :

<https://martinfowler.com/articles/microservices.html>





## Note :

Il ne faut pas faire d'opposition stricte webservice **vs** API **vs** microservices.

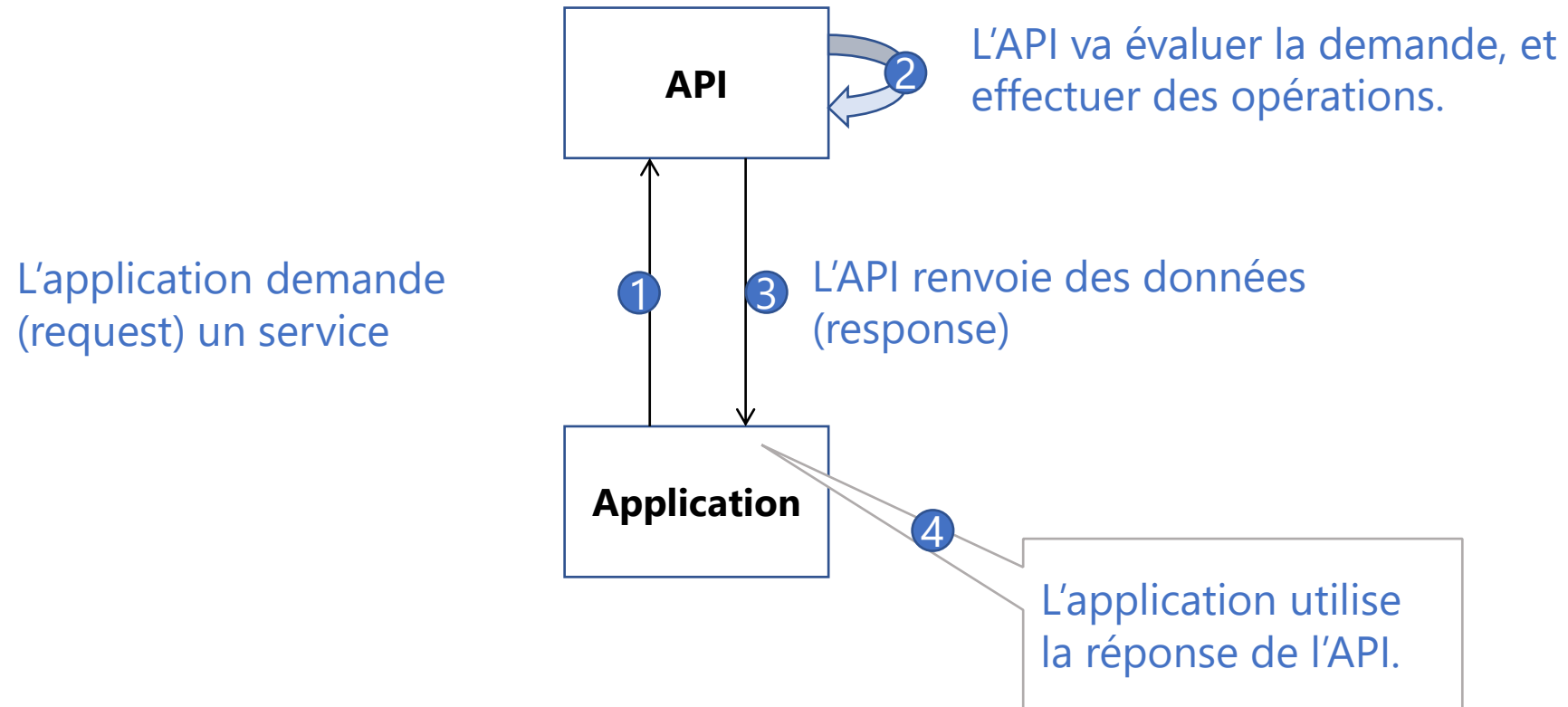
Ce sont davantage des approches philosophiques que des paradigmes technologiques mutuellement exclusifs.

Source : <https://netflixtechblog.com/engineering-trade-offs-and-the-netflix-api-re-architecture-64f122b277dd>

# Interagir avec une API

---

- I. Le modèle réponse-requête
- II. Postman : un outil
- III. *Endpoint* et méthode



### API Rest

Paramètre de la requête directement dans l'uri

GET <http://api-url/weather?city=Montreal&country=Canada>



CLIENT



SERVEUR

```
{  
  "weather": [  
    {  
      "id": 803,  
      "main": "Clouds",  
      "description": "broken clouds",  
      "icon": "04d"  
    }  
  ],  
  "base": "stations",  
  "main": {  
    "temp": 286.25,  
    "temp_min": 286.25,  
    "temp_max": 286.25,  
    "pressure": 1017.38,  
    "sea_level": 1025.23,  
    "grnd_level": 1017.38,  
    "humidity": 67  
  },  
}
```

Réponse du serveur en JSON.  
SANS ENVELOPPE !

Le contenu du message transporté  
est appelé le « payload »



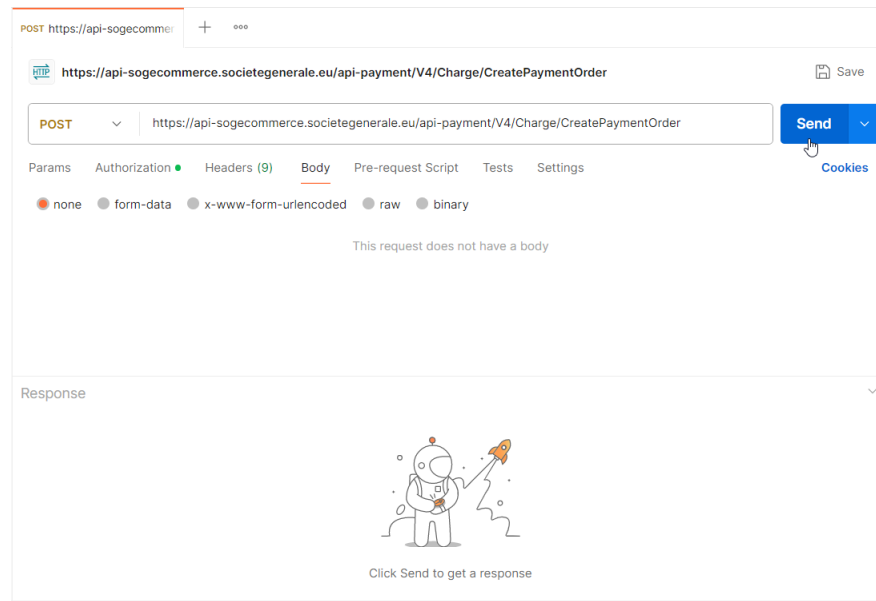
POSTMAN

Les APIs sont pensées pour être utilisées par des applications, pas par des personnes. Elles n'ont donc pas d'écrans à proprement parler.

Postman est un outil qui va nous permettre d'interagir avec elles.

Téléchargez et installez l'outil :

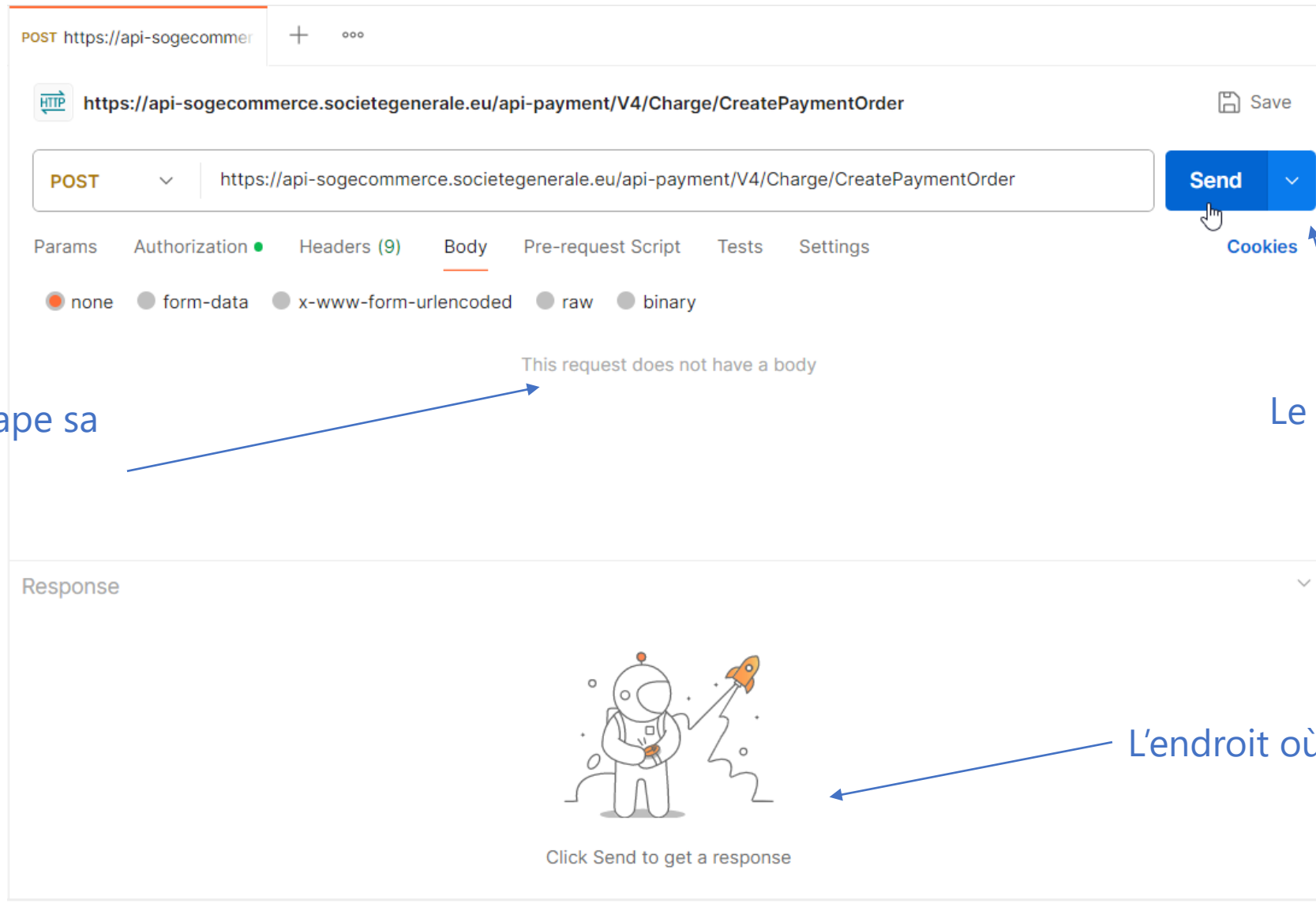
<https://www.postman.com/>



# Interagir avec une API

## Postman : un outil

L'endroit où on tape sa requête



Le bouton d'envoi

L'endroit où reçoit la réponse



Open Food Facts est une association qui entretient une [base de données collaborative](#) sur la composition des aliments.

Cette base de données est accessible à travers l'IHM du site internet <https://world-fr.openfoodfacts.org>, mais aussi à travers une API notamment utilisée par l'application Yuka.

Sélectionner GET

Taper l'adresse

The screenshot shows the Postman interface with a GET request configured. The URL bar contains `https://world.openfoodfacts.org/api/v2/product/` followed by a text box labeled **Code-barres**. Below the URL bar, the **Authorization** tab is selected, showing **Basic Auth** as the type. The **Username** field contains `fe-off` and the **Password** field contains `'aZ)X4vJnB'%t;t`. A blue arrow points from the text Renseigner les identifiants to the password field. At the bottom right, there is a **Send** button and a keyboard shortcut `Ctrl+↵`. An inset image of a UNICEF product label with a barcode is also visible.

GET | `https://world.openfoodfacts.org/api/v2/product/` **Code-barres**

Params | **Authorization** • | Headers (9) | Body | Pre-request Script | Tests | Settings

Type: Basic Auth ▼

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username: `fe-off`

Password: `'aZ)X4vJnB'%t;t` ⚠

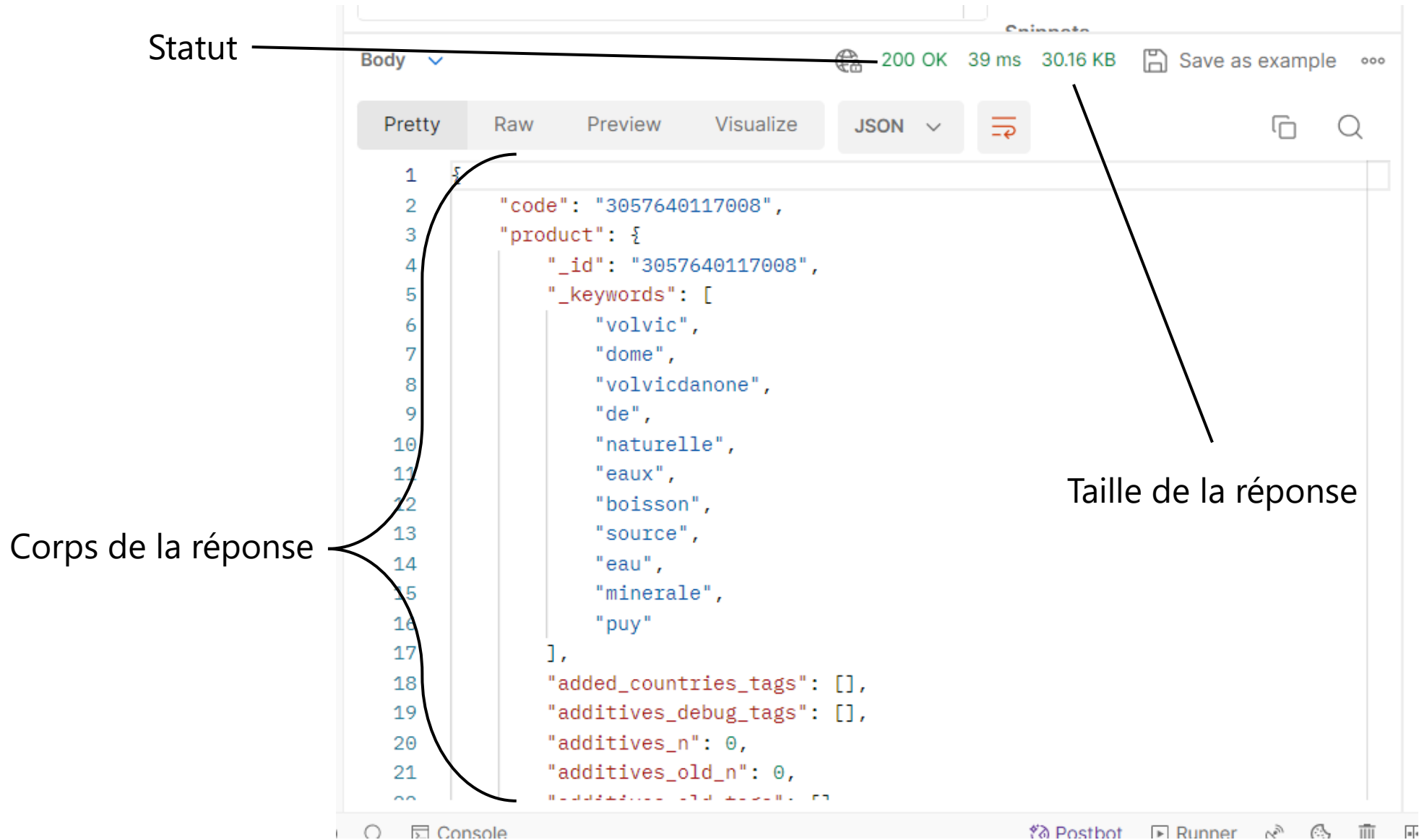
Renseigner les identifiants

Enfin, renseigner les chiffres du code-barres de n'importe quel produit alimentaire à la place de **Code-barres** et cliquer sur « SEND ».

Send ▼

Ctrl+↵ | Cookies





La **réponse** de l'API est dans un format appelé le **JSON**.

C'est un format de données structuré qui est un standard de fait dans le monde des API.

```
1  {  
2    "code": "3057640117008",  
3    "product": {  
4      "_id": "3057640117008",  
5      "_keywords": [  
6        "volvic",  
7        "dome",  
8        "volvicdanone",  
9        "de",  
10       "naturelle",  
11       "eaux",  
12       "boisson",  
13       "source",  
14       "eau",  
15       "minerale",  
16       "puy"  
17     ],  
18   },  
19   }
```

Les accolades indiquent que ce qui suit appartient à cette catégorie

L'une des caractéristiques de mon produit est son identifiant « \_id »

Si ce format est « relativement » analysable par un être humain, n'oublions pas qu'il est avant tout pensé pour que des applications puissent échanger des informations.

```
1  {  
2    "code": "3057640117008",  
3    "product": {  
4      "_id": "3057640117008",  
5      "_keywords": [  
6        "volvic",  
7        "dome",  
8        "volvicdanone",  
9        "de",  
10       "naturelle",  
11       "eaux",  
12       "boisson",  
13       "source",  
14       "eau",  
15       "minérale",  
16       "puy"  
17     ],  
18   },  
19 }
```

Tout ce qui est entre crochets « [...] » correspond à une liste de valeurs.

On peut donc lire :

« Mon produit a l'identifiant **3057640117008** et correspond aux mots-clefs **volvic, dome, volvicdanone, de, naturelle...** »

L'API [Open Food Facts](#) permet aussi de faire des recherches sur les aliments à partir d'un certain nombre de caractéristiques.



GET ▼ <https://world.openfoodfacts.org/api/v2/search/>

Il faut pour cela utiliser une autre *url*

Params Auth ● Headers (11) Body Pre-req. Tests Settings

Cookies

#### Query Params

|  | Key                              | Value                              | Description                              | ... | Bulk Edit |
|--|----------------------------------|------------------------------------|------------------------------------------|-----|-----------|
|  | <input type="text" value="Key"/> | <input type="text" value="Value"/> | <input type="text" value="Description"/> |     |           |

Dans la partie « Params » vous pouvez entrer les paramètres sur lesquels vous voulez faire une recherche et leur attribuer une valeur.

Params Auth ● Headers (11) Body Pre-req. Tests Settings Cookies

### Query Params

|  | Key                              | Value                              | Description                              | ... | Bulk Edit |
|--|----------------------------------|------------------------------------|------------------------------------------|-----|-----------|
|  | <input type="text" value="Key"/> | <input type="text" value="Value"/> | <input type="text" value="Description"/> |     |           |

Les différents paramètres sur lesquels vous pouvez faire une recherche :

additives\_tags  
allergens\_tags  
brands\_tags

categories\_tags  
labels\_tags  
nutrition\_grades\_tags

stores\_tags

Chacun de ces paramètres peut accepter plusieurs valeurs. Par exemple *additives\_tag* peut accepter *e330*, *e322*, *e322i*, *e500*, *e415*, *e471*, *e202*, *e407*, *e250*, *e133*, *e422*, *e414*...

Les paramètres de recherche :

|                |                       |             |
|----------------|-----------------------|-------------|
| additives_tags | categories_tags       | stores_tags |
| allergens_tags | labels_tags           |             |
| brands_tags    | nutrition_grades_tags |             |



5 min

Combien de produits de la marque  
*ferrero* font partie de la catégorie  
*chocolate*?

Combien de produits de la marque *ferrero* font partie de la catégorie *chocolate*?

GET ▼ [https://world.openfoodfacts.org/api/v2/search/?brands\\_tags=ferrero&categories\\_tags=chocolate](https://world.openfoodfacts.org/api/v2/search/?brands_tags=ferrero&categories_tags=chocolate)

Params ● Auth ● Headers (11) Body Pre-req. Tests Settings

| <input type="checkbox"/>            | Key             | Value     | Description |
|-------------------------------------|-----------------|-----------|-------------|
| <input checked="" type="checkbox"/> | brands_tags     | ferrero   |             |
| <input checked="" type="checkbox"/> | categories_tags | chocolate |             |

```
1 {
2   "count": 124,
3   "page": 1,
4   "page_count": 24,
5   "page_size": 24,
6   "products": [
7     {
8       "_id": "80310891",
9       "_keywords": [ ...
30     ],
31     "abbreviated_product_name": "Kinder joy t1 permanent",
32     "abbreviated_product_name_fr": "Kinder joy t1 permanent",
33     "abbreviated_product_name_fr_imported": "Kinder joy t1 permanent",
```

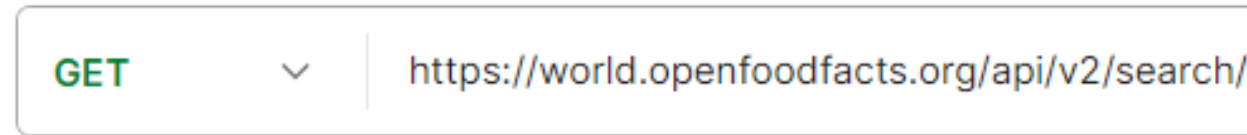
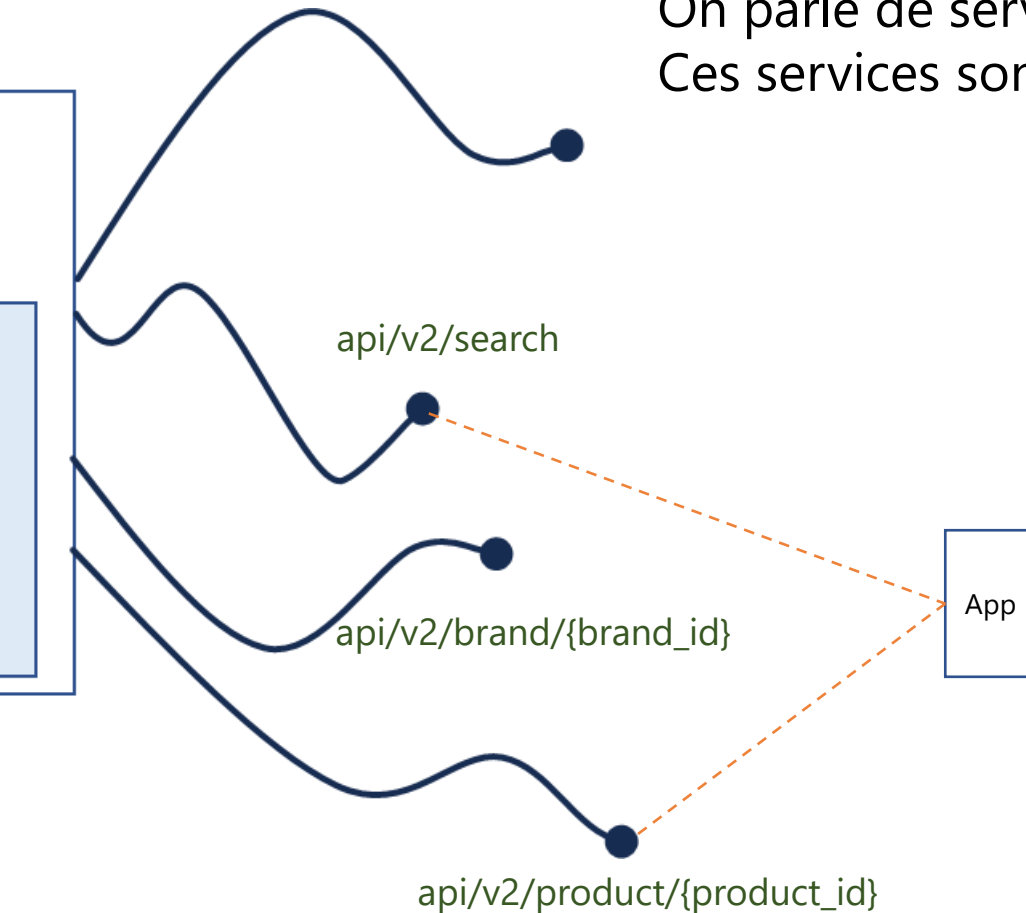


45mn

1. Combien de chocolats ferrero contiennent du e322?
2. Combien de produits vendus à monoprix sont à la fois bio (organic) et avec une note nutritive e?
3. Citer un produit qui appartient à la fois aux marques lu, milka et mondelez.
4. Quel est le produit panzani vendu à lidl avec le pire nutriscore?
5. Combien de biscuits lu contiennent du lait (milk), du gluten et des œufs (eggs)?
6. Parmi ceux-ci, combien contiennent du sorbate de potassium?
7. De quelle couleur est l'étiquette de la bière made in France vendue à leclerc et dont le nom contient le mot « Orge »?
8. Quelle est l'adresse du service consommateur panzani?



Rappel : une API, c'est la mise à disposition de services pour des applications. On parle de services **exposés** à travers une API. Ces services sont exposés via des url, aussi appelées **endpoint**.

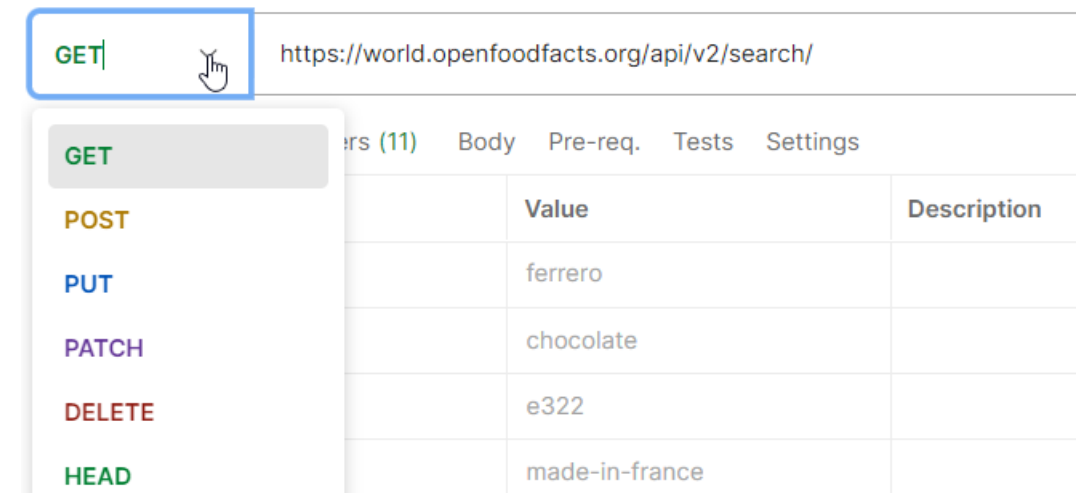
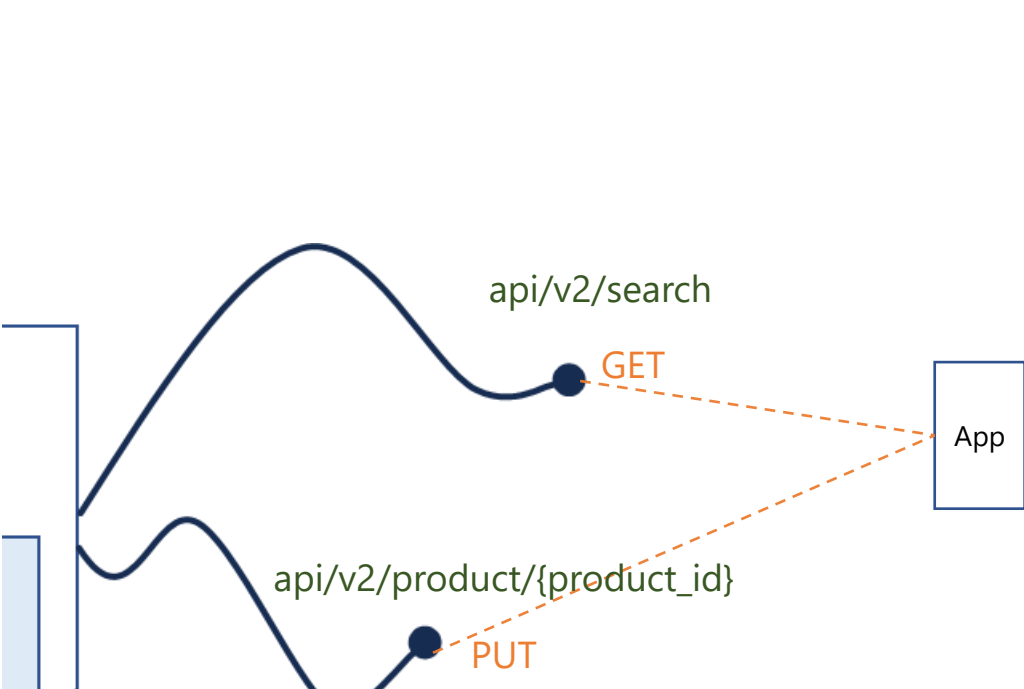


L'application va donc se connecter à un **endpoint**, comme le feraient Postman ou un navigateur internet comme Firefox.

Cette connexion se fait en utilisant le standard à la base d'internet, le [HTTP](#).

Ce [protocole](#) de communication a prévu différentes manières d'interroger une ressource.

Le client peut par exemple demander au serveur de consulter une ressource en lecture, décider d'envoyer des données, voire même d'effacer une ressource.



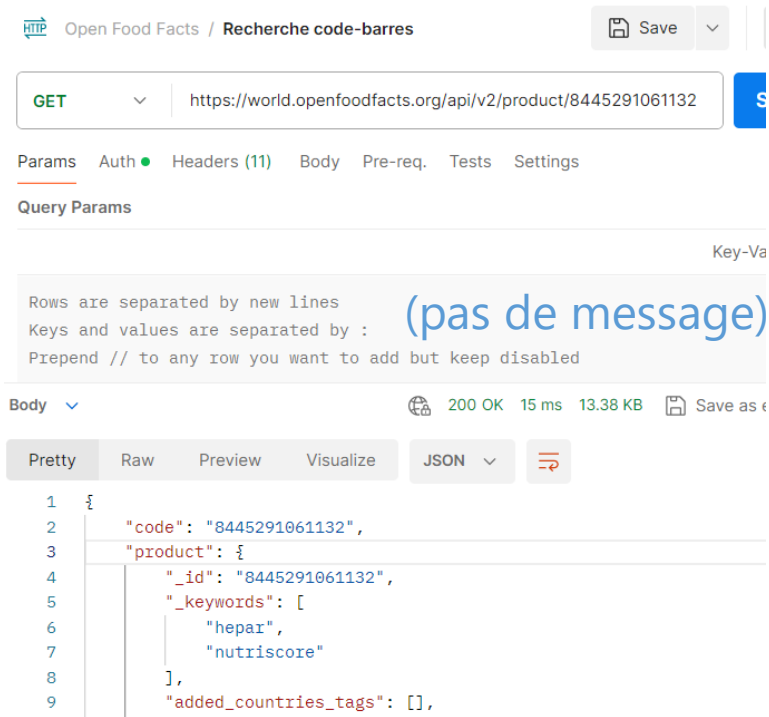
Dans Postman, il est possible de choisir comment on va interagir avec un endpoint

On appelle ces types d'interactions avec les endpoint des **méthodes**.

Les méthodes principales sont :

- **GET** – demande la ressource sans l'altérer
- **POST** – transmet des données vers la ressource
- **PATCH** – modifie les données de la ressource
- **DELETE** – efface la ressource du serveur

Une requête est toujours constituée de la méthode et de l'url de la ressource à atteindre (endpoint).



Open Food Facts / Recherche code-barres

GET https://world.openfoodfacts.org/api/v2/product/8445291061132

Params Auth Headers (11) Body Pre-req. Tests Settings

Query Params

Key-Value

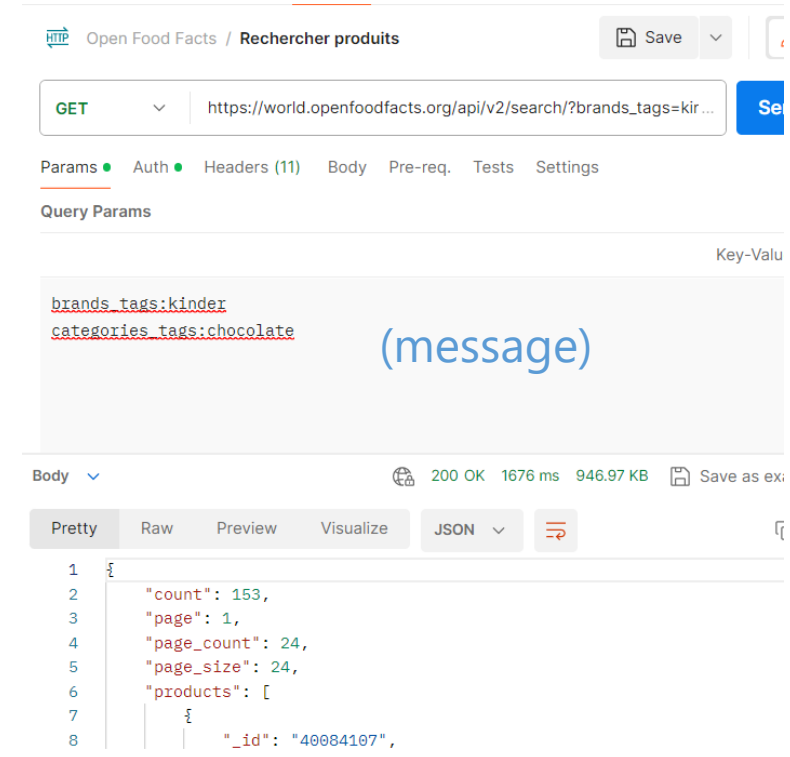
Rows are separated by new lines  
Keys and values are separated by :  
Prepend // to any row you want to add but keep disabled

(pas de message)

Body 200 OK 15 ms 13.38 KB Save as ex

Pretty Raw Preview Visualize JSON

```
1 {
2   "code": "8445291061132",
3   "product": {
4     "_id": "8445291061132",
5     "_keywords": [
6       "hepar",
7       "nutriscore"
8     ],
9     "added_countries_tags": [],
```



Open Food Facts / Rechercher produits

GET https://world.openfoodfacts.org/api/v2/search/?brands\_tags=kir...

Params Auth Headers (11) Body Pre-req. Tests Settings

Query Params

Key-Value

brands\_tags:kinder  
categories\_tags:chocolate

(message)

Body 200 OK 1676 ms 946.97 KB Save as ex

Pretty Raw Preview Visualize JSON

```
1 {
2   "count": 153,
3   "page": 1,
4   "page_count": 24,
5   "page_size": 24,
6   "products": [
7     {
8       "_id": "40084107",
```

Elle peut (ou pas) comporter un message (payload en anglais)



1 - Décrivez le résultat de ces requêtes :

- **POST** `http://monblog.com/photos`
- **DELETE** `http://monblog.com /photos/200`
- **GET** `http://monblog.com /photos/31/commentaires`
- **PATCH** `http://monblog.com /photos/2/commentaires/5`

2 - Lesquelles sont accompagnées d'un payload avec la requête ? Dans la réponse ?

Après avoir requêté un endpoint, le client va recevoir une réponse accompagnée d'un **statut** – qui correspond à un **code HTTP**.



- 200 – requête traitée en succès
- 201 – requête traitée en succès, document créé
- 400 – requête erronée
- 401 – requête non permise (authentification)
- 404 – ressource non trouvée (la plus connue)
- 405 – méthode non autorisée
- 500 – erreur interne du serveur
- 504 – temps d'attente écoulé

Voir aussi : <https://http.cat/>

# Documenter, concevoir, spécifier

- I. Lire une documentation
- II. Concevoir et documenter



5 min

Quelles difficultés avez-vous  
rencontrées lors de l'exercice avec  
Postman?



Les API sont pensées pour permettre à des machines d'interagir entre elles, sous des formats définis à l'avance.

Sans connaître les paramètres qui existent, comment ils sont utilisés, comment ils sont organisés entre eux, et les valeurs qu'ils peuvent prendre, il va être difficile d'interagir avec l'application.

|                                     |             |                      |             |
|-------------------------------------|-------------|----------------------|-------------|
| <input checked="" type="checkbox"/> | departement | <input type="text"/> |             |
|                                     | Key         | Value                | Description |

Faut-il taper « Ain », « 01 », « 1 »  
ou « Ressources Humaines »?

Est-ce que le système accepte n'importe quel contenu ou attend des options identifiées à l'avance?

Chaînes de caractères, nombre, booléens?

Les API sont donc accompagnées par une **documentation**, qui va décrire les services offerts et la manière d'interagir avec eux.

Observez les différentes informations  
présentes sur cette page.

Exemple : <https://pole-emploi.io/data/api/pole-emploi-connect/experiences-professionnelles>

## Expériences professionnelles <sup>v1</sup>

🔒 Accès conditionné

Avec l'API Expériences professionnelles, accédez aux expériences professionnelles déclarées par un utilisateur sur pole-emploi.fr. Découvrez vite les caractéristiques, la documentation et les conditions d'obtention et d'exploitation de cette API.

Demandeur d'emploi

Présentation

Documentation

Dispositif Pôle emploi Connect

### Présentation

L'API Expériences professionnelles vous permet d'accéder aux expériences saisies par une personne dans son profil de compétences sur le site pole-emploi.fr.

Ces données sont également utilisées pour le système de rapprochement entre les CV et les offres du site Pôle emploi.

Les informations obtenues sont notamment :

- Fonction
- Nom de l'entreprise
- Périodes de travail

Documentation technique

- Caractéristiques
- Récupérer la liste des expériences professionnelles

Caractéristiques

|                                                 |                                                                 |
|-------------------------------------------------|-----------------------------------------------------------------|
| Mode d'accès                                    | restreint (demande d'accès soumise à validation de Pôle emploi) |
| Fréquence de mise à jour                        | temps réel                                                      |
| Cinématique OAuth                               | <i>authorization code flow</i>                                  |
| Royaume Pôle Emploi Access Management /individu |                                                                 |

Documentation technique

- Caractéristiques
- Récupérer la liste des expériences professionnelles

Récupérer la liste des expériences professionnelles

Cette API vous permet d'accéder aux expériences saisies par un individu dans son profil de compétences sur pole emploi.fr : fonction, nom de l'entreprise, périodes de travail etc. Ces données sont également utilisées pour le système de rapprochement entre les CV et les offres sur pôle emploi.fr

Description de la requête

Point d'accès

Cette ressource permet de récupérer la liste des expériences saisies dans son profil de compétences par un individu

Comment utiliser les API ? →

Documentation technique

- Caractéristiques
- Récupérer la liste des expériences professionnelles

Récupérer la liste des expériences professionnelles

Cette API vous permet d'accéder aux expériences saisies par un individu dans son profil de compétences sur pole emploi.fr : fonction, nom de l'entreprise, périodes de travail etc. Ces données sont également utilisées pour le système de rapprochement entre les CV et les offres sur pôle emploi.fr

Exemple d'appel

```
GET https://api.pole-emploi.io/partenaire/peconnect-experiences/v1/experiences
Authorization: Bearer [Access token]
```

Données retournées

| Code        | Cardinalité | Format   | Description                                                              |
|-------------|-------------|----------|--------------------------------------------------------------------------|
| date.debut  | 0,1         | DateTime | Date de début de l'expérience                                            |
|             |             |          | Exemple : 2014-01-01T00:00:00+01:00                                      |
| date.fin    | 0,1         | DateTime | Date de fin de l'expérience                                              |
|             |             |          | Exemple : 2017-02-01T00:00:00+01:00                                      |
| description | 0,1         | String   | Description de l'expérience si elle est renseignée.                      |
| duree       | 0,1         | Long     | Durée exprimée en nombre de mois de l'expérience si elle est renseignée. |
|             |             |          |                                                                          |
| enPoste     | 1           | Boolean  | Booléen indiquant si l'expérience est toujours en cours.                 |
|             |             |          | Remarque : un individu peut avoir plusieurs expériences en cours         |
| entreprise  | 0,1         | String   | Nom de l'entreprise s'il est renseigné                                   |

Exemple de retour

```
HTTP 200 OK
Content-Type: application/json
Cache-Control: no-cache
[
  {
    "date": {
      "debut": "2018-02-01T00:00:00+01:00"
    },
    "description": "service de 150 couverts, encadrement d'une équipe de",
    "duree": 5,
    "enPoste": true,
    "entreprise": "Hippopotamus",
    "etranger": false,
    "intitule": "serveur",
    "lieu": "Nantes"
  },
  {
    "date": {
      "debut": "2011-06-01T00:00:00+02:00"
    },
    "duree": 85,
    "enPoste": true,
    "etranger": false,
    "intitule": "Engagement associatif"
  },
  {
    "date": {
      "debut": "2014-01-01T00:00:00+01:00",
      "fin": "2015-01-01T00:00:00+01:00"
    },
    "description": "travail sur les chantiers",
    "duree": 13,
    "enPoste": false,
    "entreprise": "Hippopotamus"
```



## Web service Charge/CreatePaymentOrder

Adresse documentation : <https://sogecommerce.societegenerale.eu/doc/fr-FR/rest/V4.0/api/playground/Charge/CreatePaymentOrder>



20 min

En utilisant la documentation, effectuer un paiement de *125 dinars tunisiens* pour la commande *REF-COMM0903* via Postman

Params Authorization ● Headers (12)

Type Basic Auth ▼

Username : 61881992

Password : testpassword\_q4JtCk2nuiMsl2hi1uU394oqQ5dB1sOoMMaS4k8SaiY00

Params Authorization ● Headers (12) Body Pre-request S

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ b

1

Il va falloir renseigner un *payload* dans l'onglet « Body »

Web service  
Charge/CreatePaymentOrder

POST

<https://api-sogecommerce.societegenerale.eu/api-payment/V4/Charge/CreatePaymentOrder>

```
1 {  
2   · "amount": 125000,  
3   · "currency": "TND",  
4   · "orderId": "REF-COMM0903"  
5 }
```

POST

<https://api-sogecommerce.societegenerale.eu/api-payment/V4/Charge/CreatePaymentOrder>

Send

Paramètres principaux

tout montrer

amount

requis

currency

requis

orderId

recommandé

> channelOptions

formAction

On renseigne les 3 paramètres qui nous intéressent au format JSON. On peut voir qu'ils sont tous au même niveau.

```
1 {  
2   · "amount":  
3   · "currency":  
4   · "orderId":  
5 }
```

Note : les paramètres sont séparés par une virgule

currency

Devise du paiement. Code alphabétique en majuscule selon la norme ISO 4217 alpha-3.

Exemple: "EUR" pour l'euro.

Format

|                   |                                   |
|-------------------|-----------------------------------|
| Type              | chaîne                            |
| Exemple           | EUR                               |
| Longueur minimale | 3                                 |
| Longueur maximale | 3                                 |
| Requis            | Ce paramètre est requis           |
| Peut être nul ?   | Ce paramètre ne peut pas être nul |

| DEVISE                         | CODIFICATION<br>ISO 4217 | UNITÉ<br>FRACTIONNAIRE |
|--------------------------------|--------------------------|------------------------|
| Couronne suédoise (752)        | SEK                      | 2                      |
| Baht thaïlandais (764)         | THB                      | 2                      |
| Dinar Tunisien (788)           | TND                      | 3                      |
| Lire turque (949)              | TRY                      | 2                      |
| Nouveau dollar de Taïwan (901) | TWD                      | 2                      |
| Dollar des États-Unis (840)    | USD                      | 2                      |
| Rand sud-africain (710)        | ZAR                      | 2                      |

Le paramètre *currency* correspond à la devise du paiement, en majuscule, sur 3 caractères. On renseigne la devise qui nous intéresse.

Note : les chaînes de caractères (string) sont entourées par des guillemets.

```
1  {
2  |  · "amount": ·
3  |  · "currency": · "TND",
4  |  · "orderId": ·
5  }
·
```



amount

Montant du paiement dans sa plus petite unité monétaire (le centime pour l'euro).

Exemple: 30050 pour 300,50 EUR.

Format

|                   |                                   |
|-------------------|-----------------------------------|
| Type              | entier                            |
| Exemple           | 990                               |
| Longueur minimale | 1                                 |
| Longueur maximale | 12                                |
| Requis            | Ce paramètre est requis           |
| Peut être nul ?   | Ce paramètre ne peut pas être nul |

Le montant du paiement est décrit comme étant un nombre entier sur un à 12 caractères, dans sa plus petite unité monétaire.

| DEVISE               | CODIFICATION ISO 4217 | UNITÉ FRACTIONNAIRE |
|----------------------|-----------------------|---------------------|
| (704)                |                       |                     |
| Dinar Tunisien (788) | TND                   | 3                   |
|                      |                       |                     |

Celle du dinar étant sur trois chiffres après la virgule, on renseigne donc 125000 pour 125 dinars.

```
1 {
2   "amount": 125000,
3   "currency": "TND",
4   "orderId": "REF-COMM0903"
5 }
```

Documenter, concevoir, spécifier

# Lire une documentation : corrigé

POST

https://api-sogecommerce.societegenerale.eu/api-payment/V4/Charge/CreatePayme...

12

```
{
  "amount": 125000,
  "currency": "TND",
  "orderId": "REF-COMM0903"
}
```

Body

Cookies (1)

Headers (11)

Test Results

200 OK723 ms1.47 KB

Save as

Pretty

Raw

Preview

Visualize

JSON

1

{

2

"webService": "Charge/CreatePaymentOrder",

3

"version": "V4",

4

"applicationVersion": "6.11.1",

5

"status": "SUCCESS",

6

"answer": {

7

"paymentOrderId": "fc38698c29d04b02a13af7703ef4f0c7",

8

"paymentURL": "https://sogecommerce.societegenerale.eu/t/68rf0a4t",

9

"paymentOrderStatus": "RUNNING",

10

"creationDate": "2024-01-26T09:48:51+00:00",

11

"updateDate": null,

12

"amount": 125000,

13

"currency": "TND",

14

"locale": "fr\_FR",

15

"strongAuthentication": "AUTO",

16

"orderId": "REF-COMM0903",

17

"channelDetails": {

18

"channelType": "URL",

L'ordre de paiement a été  
créé avec succès!





https://sogecommerce.societegenerale.eu/doc/fr-FR/

Identifiant du marchand :

61881992

Référence commande :

REF-COMM0903

Montant :

125,000 TND  
( 36,87 EUR\* )

\* Contrevaaleur à titre indicatif



## Les différents *endpoint*

La documentation de l'API Open Food Facts est disponible ici :  
<https://openfoodfacts.github.io/openfoodfacts-server/api/ref-v2/>

OPERATIONS

Read Requests

- Get information for a specific product by barcode
- Get Knowledge panels for a specific product by barcode (special case of get product)
- Performing OCR on a Product
- Search for Products
- Get Suggestions to Aid Adding/Editing Products

Write Requests

- Add a Photo to an Existing Product
- Rotate A Photo
- Crop A Photo
- Add or Edit A Product
- Unselect A Photo

## Get information for a specific product by barcode

**GET** /api/v2/product/{barcode}

A product can be fetched via its unique barcode. It returns all the details of that product response.

### REQUEST

#### PATH PARAMETERS

\* barcode string

The barcode of the product to be fetched  
**Examples:** 3017620422003

API Server <https://world.openfoodfacts.net>  
Authentication Not Required

FILL EXAMPLE

CLEAR

TRY

### RESPONSE

200

OK

EXAMPLE SCHEMA

application/json

#### OBJECT

Multiline description

```
{
  code: string
  status: integer
  status_verbose: string
  product: {
    abbreviated_product_name: string
    code: string
    codes_tags: [string]
```

Barcode of the product (can be EAN-13 or internal

This is all the fields describing a product and how to  
Abbreviated name in requested language  
barcode of the product (can be EAN-13 or internal  
→ [ A value which is the type of barcode "code-13" or



45 mn



Félicitations! Vous venez de rejoindre l'équipe d'Open Food Facts.

Une demande fréquente de vos clients est l'amélioration des fonctionnalités liées aux allergies. En effet, aujourd'hui, il n'est pas possible de faire une recherche sur un ou plusieurs produits qui ne contiennent pas un allergène.



45 mn

En petits groupes, discutez de comment vous pourriez faire évoluer l'API pour offrir aux applications de vos clients un nouvel *endpoint* qui réponde à leurs besoins.

Une fois que vous avez décidé de ce que vous voulez faire, documentez votre nouvel endpoint en vous inspirant de la documentation existante.

Donnez un nom à votre service, une méthode et un endpoint, une description générale et décrivez les paramètres de recherche.

## READ REQUESTS

### Search for Products without Allergens

**GET** /api/v2/searchWithoutAllergens

La recherche de produits sans allergènes permet d'obtenir la liste des produits d'une catégorie qui ne contiennent pas le/les allergènes recherchés. On peut utiliser plusieurs valeurs pour un même paramètre de recherches en les séparant par une virgule(,).

## PARAMETRES DE RECHERCHE

**categories\_tag** (string) **requis**

La catégorie de produit(s) recherché(s). Cf. liste des valeurs possibles.

**Exemple** : yoghurt

**allergens\_tag** (string) **requis**

Les allergènes à exclure de la recherche. Cf. liste des valeurs possibles.

**Exemple** : milk

**brands\_tag** (string) **optionnel**

La marque des produits recherchés. Cf. liste.

**Exemple** : buitoni